Major Project Report

CodeQuest - Adaptive Debug-Based Coding Learning Platform

Project Category: University Based Project

Projexa Team Id- 26E4004

Submitted in partial fulfilment of the requirement of the degree of

BACHELORS OF COMPUTER APPLICATION

Semester-VI

to

K.R Mangalam University

by

Ayush Pandey (2301201015) Section-A
Kundan Yadav (2301201026) Section-A

Under the supervision of

Dr. Manish Kumar
Assistant Professor

Department of Computer Science and Engineering
School of Engineering and Technology
K.R Mangalam University, Gurugram- 122001, India

May 2026

DECLARATION

I hereby declare that this project report is my original work and has not been submitted elsewhere. The information presented here is true and correct to the best of my knowledge.

Ayush Pandey
(2301201015)

Kundan Yadav
(2301201026)

Date

CERTIFICATE OF UNIVERSITY-BASED PROJECT COMPLETION

This is to certify that the students listed below have **successfully completed** a project based on a **University-based problem**, undertaken as part of their academic or innovation-related engagement at **K. R. Mangalam University**.

Title of the Project:
"**CodeQuest - Adaptive Debug-Based Coding Learning Platform**"

Name(s) of Student(s):

- Ayush Pandey – Roll No: 2301201015
- Kundan Yadav – Roll No: 2301201026

Duration:
From **January 2026** to **May 2026**

This is to acknowledge that the above-mentioned project has been **successfully completed** and has effectively addressed a **real-world problem within the university ecosystem**.
The initiative is appreciated for its **innovation, practical relevance, and contribution** toward institutional improvement and **student-led problem solving**.

We congratulate the project team for their dedication, creativity, and successful completion of this university-based initiative.

Signature: _____
Dr. Manish Kumar

Date:

School of Engineering & Technology

K. R. Mangalam University

CERTIFICATE

This is to certify that the Project Synopsis entitled, "**CodeQuest - Adaptive Debug-Based Coding Learning Platform**" submitted by "**Ayush Pandey(2301201015) & Kundan Yadav (2301201026)"** to **K.R Mangalam University, Gurugram, India,** is a record of bonafide project work carried out by them under my supervision and guidance and is worthy of consideration for the partial fulfilment of the degree of **Bachelors Of Computer Application** of the University.

Dr. Manish Kumar
Assistant Professor

Date: 20th April 2026

INDEX

S No	Content	Page No
1.	Project Completion Certificate	ii
2.	Abstract	1
3.	Introduction	2-5
4.	Motivation	6-7
5.	Literature Review/Comparative work evaluation	8-10
6.	Gap Analysis	11-13
7.	Problem Statement	14
8.	Objectives	15-16
9.	Methodology	17-24
10.	Tools/platform Used	25-26
11.	Environmental Setup	27-28
12.	Implementation	29-32
13.	Results And Discussion	33-35
14.	Conclusion & Future Work	36-37
15.	References	38
16.	Annexure I: Plagiarism Declaration Certificate	39
17.	Annexure II Plagiarism Report	40

ABSTRACT

Programming education plays a crucial role in developing problem-solving, logical reasoning, and analytical thinking skills among university students. However, many learners face challenges in mastering programming concepts due to limited practical exposure, lack of personalized learning support, and minimal focus on real-world debugging scenarios. Conventional classroombased teaching and linear learning platforms often fail to adapt to individual learning gaps, resulting in reduced comprehension and confidence among students.

CodeQuest is a web-based adaptive coding learning platform designed to address these challenges by emphasizing learning through debugging. Instead of focusing solely on writing code from scratch, the platform presents students with intentionally broken code snippets that simulate common programming errors. Students are required to analyze, debug, and correct the code, thereby strengthening their understanding of programming logic, syntax, and execution flow.

Additionally, CodeQuest provides a centralized dashboard for tracking student progress, enabling learners to monitor their improvement over time. Faculty members can leverage analytical insights from the platform to identify common learning challenges, evaluate overall class performance, and provide focused academic support. By combining adaptive learning, debugging-based practice, and performance analytics, CodeQuest aims to enhance programming proficiency, improve logical thinking, and create an effective, self-paced learning environment for university students.

KEYWORDS: Competitive Programming, Coding Platform, Problem Solving, Web Application, Learning System

Chapter 1
Introduction

1. Background of the project

In the modern digital era, programming has become a fundamental skill across various domains such as software engineering, data science, artificial intelligence, and web development. Educational institutions emphasize programming education as a core component of computer science and information technology curricula. The ability to understand programming concepts, write efficient code, and solve computational problems is essential for students who aim to build careers in technology-driven industries.

Despite the growing importance of programming skills, many university students struggle to grasp core programming concepts. These challenges often arise due to limited practical exposure, insufficient debugging practice, and the lack of personalized learning environments. Traditional teaching methodologies generally follow a lecture-based approach where instructors introduce programming concepts through theoretical explanations and simple examples. Although this method provides a basic understanding of syntax and logic, it does not adequately prepare students to deal with real-world programming challenges.

One of the most important skills in programming is **debugging**, which involves identifying and correcting errors in code. In real-world software development, programmers spend a significant amount of time analyzing and fixing faulty code rather than writing programs from scratch. However, debugging is rarely emphasized as a primary learning strategy in conventional programming education. As a result, students may understand programming

syntax but struggle to identify logical errors, runtime issues, and unexpected program behavior.

To address these challenges, modern educational technologies are increasingly focusing on **interactive and adaptive learning platforms**. These platforms provide hands-on practice, automated evaluation, and personalized learning paths to help students improve their programming skills through continuous engagement.

CodeQuest is proposed as a **web-based adaptive coding learning platform** designed to enhance programming education through a debug-based learning approach. Instead of simply solving coding problems, students are presented with intentionally broken code snippets that simulate real programming errors. Learners must analyze the code, identify mistakes, and correct them to produce the correct output. This approach helps students understand programming logic more deeply and develop critical debugging skills.

The platform also integrates features such as automated code evaluation, adaptive difficulty levels, progress tracking, and gamification elements like points, levels, and badges. These features encourage consistent practice, improve engagement, and allow learners to track their performance over time.

In addition, CodeQuest provides analytical insights for instructors and educators. Faculty members can monitor student progress, identify common mistakes, and provide targeted academic support to improve learning outcomes.

By combining **debug-based learning, adaptive problem difficulty, and performance analytics**, CodeQuest aims to create a modern, interactive, and effective programming learning environment that helps students develop strong coding and problem-solving skills.

Table 1. Existing systems

Factors	**Evaluation Criteria**	**Platform A (LeetCode)**	**Platform B (HackerRank)**	**Platform C (Traditional Lab Learning)**
Learning Approach	Focus on debugging practice	Limited	Limited	Very Limited
Problem Difficulty	Beginner to Advanced	Advanced oriented	Beginner to Advanced	Basic level
Automated Code Evaluation	Testcase based evaluation	Yes	Yes	No
Adaptive Learning	Difficulty adjusts based on performance	No	Partial	No
Real-world Debugging Scenarios	Brokencode correction	Rare	Rare	Not included
Gamification	Points, badges, ranking	Yes	Yes	No
Progress Tracking	Dashboard and analytics	Yes	Yes	Limited

Faculty Monitoring	Instructor insights	No	Limited	Yes
User Engagement	Interactive coding interface	High	High	Low
Learning Flexibility	Self-paced learning	Yes	Yes	Limited

Table 2. Key Features Comparison

Feature	**Existing Coding Platforms**	**CodeQuest**
Debug-Based Learning	Not a primary focus	Core learning method
Adaptive Difficulty	Limited	Fully adaptive
Automated Feedback	Available	Available with debugging hints
Student Progress Analytics	Basic	Detailed dashboards
Faculty Monitoring	Limited	Full analytics support
Gamification	Points and rankings	Points, levels, badges, streaks
Curriculum Alignment	Not always aligned with universities	Designed for academic learning

2. MOTIVATION

The rapid growth of the software industry and the increasing dependence on digital technologies have created a high demand for skilled programmers. Organizations today require developers who possess not only theoretical knowledge of programming languages but also strong problem-solving and debugging skills. However, many university students struggle to develop these practical abilities due to limitations in traditional teaching methodologies.

In most academic environments, programming education focuses primarily on syntax, theoretical concepts, and writing programs from scratch. While these approaches help students understand basic programming constructs, they often fail to teach students how to analyze, debug, and correct faulty code. In real-world software development, debugging is one of the most critical activities, as developers frequently work on maintaining and fixing existing codebases rather than building applications entirely from the beginning.

Another challenge faced by students is the lack of personalized learning systems. Traditional classroom teaching follows a uniform pace that may not suit every learner. Some students require additional practice and guidance to fully understand programming concepts, while others may progress faster and seek more challenging tasks. Without adaptive learning mechanisms, many students lose motivation or struggle to keep up with the course.

Existing online coding platforms provide programming challenges and automated evaluation systems; however, many of these platforms are designed primarily for competitive programming or experienced programmers. Beginners often find them difficult to use because they lack structured guidance and debugging-focused exercises.

The motivation behind developing **CodeQuest** is to address these challenges by creating a learning platform that emphasizes **debug-based learning**, adaptive difficulty, and interactive engagement. By presenting intentionally broken code snippets and asking students to identify and fix errors, the platform encourages learners to understand program behavior, analyze logical mistakes, and strengthen their debugging skills.

Additionally, CodeQuest incorporates features such as automated feedback, performance tracking, and gamification elements including points, levels, and badges. These features help maintain learner motivation and encourage consistent practice.

Overall, the motivation for this project is to create an **interactive, adaptive, and student-centric coding platform** that bridges the gap between theoretical programming education and real-world software development practices, ultimately helping students become more confident and skilled programmers.

Chapter 2

LITERATURE REVIEW

1. Review of existing literature

Programming education has been an active research area due to the increasing demand for skilled software developers and the challenges faced by students while learning programming concepts. Several studies have highlighted that traditional lecture-based teaching methods are often insufficient for developing strong programming and problem-solving abilities. As a result, many researchers have explored alternative approaches such as interactive learning platforms, automated coding systems, adaptive learning environments, and gamified educational tools.

Many online coding platforms have been developed to help learners practice programming through problem-solving exercises. Platforms such as LeetCode, HackerRank, and Codeforces provide structured coding challenges and automated evaluation systems. These platforms allow users to submit solutions and receive instant feedback based on predefined test cases. While such platforms are highly effective for improving competitive programming skills, they are often more suitable for intermediate or advanced programmers. Beginners may struggle because these systems do not always provide sufficient guidance for understanding programming errors.

Research also shows that **debugging plays a crucial role in programming education**. Debugging helps students understand program behavior and encourages deeper cognitive engagement with the code. When students analyze incorrect code and identify logical or syntactic mistakes, they develop stronger analytical and reasoning skills compared to simply writing programs from scratch.

Another important development in modern education is the use of **adaptive learning systems**. Adaptive learning platforms dynamically adjust learning content based on the performance and progress of individual learners. By identifying a student's strengths and weaknesses, these systems recommend appropriate exercises and provide personalized

learning experiences. Studies have shown that adaptive learning significantly improves engagement and learning outcomes.

In addition, **gamification techniques** such as points, levels, badges, and leaderboards have been widely adopted in educational platforms to increase student motivation. Gamified learning environments encourage students to practice consistently and maintain interest in learning activities.

Despite these advancements, there is still a lack of integrated systems that combine **debug-based learning, adaptive difficulty mechanisms, gamification, and academic progress analytics** specifically for university-level programming education. This gap highlights the need for a platform like **CodeQuest**, which integrates these educational approaches into a single system designed to improve programming comprehension and debugging skills.

Table 2.1. LITERATURE REVIEW/COMPARITIVE WORK

Author	Year	Research Title	Key Contribution
Robins, Rountree & Rountree	2003	Learning and Teaching Programming: A Review	Identified major challenges faced by beginners in programming education.
Fitzgerald et al.	2008	Debugging from the Student Perspective	Demonstrated that debugging activities significantly improve programming comprehension.
Ihan tola et al.	2015	Educational Tools for Programming	Highlighted the importance of automated feedback systems in coding education.

Brusilovsky	2007	Adaptive Educational Systems	Introduced adaptive learning techniques that personalize learning paths.
Dicheva et al.	2015	Gamification in Education	Showed that gamification increases student engagement and motivation.

Table 2.2. COMPARISON OF LEARNING APPROACHES IN PROGRAMMING EDUCATION

Learning Approach	Description	Advantages	Limitations
Traditional Classroom Learning	Lecture-Based Teaching With Lab Exercises	Structured Learning	Limited Practical Exposure
Competitive Coding Platforms	Online Coding Challenges With Automated Evaluation	Improves Problem-Solving Skills	Difficult For Beginners
Adaptive Learning Platforms	Systems That Adjust Difficulty Based On Performance	Personalized Learning	Limited Debugging Focus
Debug-Based Learning	Learning By Analyzing And Correcting Faulty Code	Improves Analytical Thinking	Less Commonly Implemented

2. GAP ANALYSIS

The review of existing research and currently available coding learning platforms reveals several improvements in programming education through online coding environments, automated code evaluation systems, and gamified learning platforms. These systems have significantly improved access to programming practice and enabled learners to solve problems in interactive environments.

However, despite these advancements, several limitations still exist in current programming learning systems, particularly for beginner and intermediate learners in academic settings.

Most existing coding platforms focus primarily on **problem-solving through writing code from scratch**. While this approach is effective for improving algorithmic thinking, it does not adequately train students to analyze and fix errors in existing code. In real-world software development, debugging is one of the most common tasks performed by developers. The lack of structured debugging exercises in current platforms limits students' ability to understand program behavior and identify logical or runtime errors.

Another limitation is the **lack of adaptive learning mechanisms** in many platforms. Many systems provide the same set of problems to all learners, regardless of their skill level or learning progress. This one-size-fits-all approach can make learning difficult for beginners while not being challenging enough for advanced learners.

Additionally, many existing coding platforms are designed primarily for **competitive programming environments** rather than academic learning. As a result, they may not align well with university curricula or classroom teaching methods.

Another important gap is the **limited availability of analytical insights for instructors**. Most platforms focus only on individual user progress and do not provide comprehensive analytics that can help instructors understand common learning difficulties among students.

Furthermore, while some platforms include gamification features such as points, rankings, and badges, these features are often not integrated with adaptive learning or debugging-based exercises.

To address these limitations, the **CodeQuest platform** is designed to integrate debugging-based learning, adaptive problem difficulty, automated evaluation, gamification, and detailed learning analytics within a single system tailored for university-level programming education.

Table 3.1. GAP ANALYSIS OF EXISTING SYSTEMS

Feature / Aspect	**Existing Coding Platforms**	**Identified Gap**	**CodeQuest Solution**
Debugging Practice	Limited or not structured	Students lack debugging skills	Debug-based learning using faulty code
Adaptive Learning	Limited personalization	Same difficulty for all learners	Adaptive difficulty adjustment
Beginner-Friendly Learning	Often designed for competitive programmers	Difficult for beginners	Structured learning progression
Automated Feedback	Basic output validation	Limited explanation of errors	Detailed feedback and debugging hints

Faculty Monitoring	Minimal instructor insights	Hard to track student learning patterns	Analytics dashboard for instructors
Gamification	Points and rankings	Not integrated with learning progress	Gamification with learning milestones

3. PROBLEM STATEMENT

Programming education in universities often relies on traditional teaching methodologies that emphasize theoretical instruction and linear learning approaches. While these methods introduce fundamental programming concepts, they provide limited opportunities for hands-on practice, personalized learning, and real-world debugging experience. As a result, many students struggle to understand program behavior, identify logical and runtime errors, and apply concepts effectively in practical scenarios.

Existing online coding platforms primarily focus on competitive programming or advanced problem-solving, making them less suitable for beginners and intermediate learners. These platforms lack adaptive learning mechanisms, structured debugging exercises, and alignment with academic curricula. Consequently, students become dependent on manual lab assistance or external resources, which restricts independent learning and slows skill development.

There is a need for an adaptive, practice-oriented coding learning system that emphasizes debugging as a core learning strategy, personalizes learning paths based on individual performance, and provides continuous feedback. Such a system should support self-paced learning for students while offering analytical insights to faculty for monitoring progress and identifying learning gaps.

CodeQuest aims to address these challenges by providing a web-based adaptive debug-based coding learning platform that enhances programming comprehension, logical reasoning, and problem-solving skills among university students

4. OBJECTIVES

The primary objective of the **CodeQuest project** is to design and develop a web-based adaptive coding learning platform that improves programming skills through debug-based learning. The system aims to provide an interactive and practical environment where students can strengthen their understanding of programming concepts by analyzing and correcting faulty code.

The specific objectives of the project are as follows:

1. **To develop an interactive web-based coding platform** that allows students to practice programming concepts in a structured and user-friendly environment.
2. **To implement debug-based learning** by providing intentionally broken code snippets that help students understand common syntax, logical, and runtime errors.
3. **To improve students' problem-solving and analytical thinking skills** by encouraging systematic code analysis, debugging, and correction.
4. **To implement an adaptive learning mechanism** that adjusts the difficulty level of coding challenges based on the learner's performance and progress.
5. **To provide automated evaluation and real-time feedback** using predefined test cases so that students can immediately understand mistakes and improve their solutions.
6. **To support self-paced learning** so that students can practice programming independently without relying heavily on manual lab assistance.

7. **To track student progress and performance analytics** through dashboards that display solved problems, accuracy, and learning trends.
8. **To incorporate gamification elements** such as points, levels, badges, and streaks to increase learner engagement and motivation.
9. **To assist instructors and faculty members** by providing analytical insights into student performance, common programming errors, and topic-wise learning difficulties.

CHAPTER 3: METHODOLOGY

The methodology section explains the systematic approach used to design and develop the **CodeQuest – Adaptive Debug-Based Coding Learning Platform**. It describes the system architecture, development workflow, data handling process, and the technologies used for implementing the platform.

The objective of the methodology is to create an efficient and scalable coding learning system that supports debugging-based learning, adaptive difficulty adjustment, automated code evaluation, and performance tracking.

3.1 **Overall architecture /Flow chart** :

The overall architecture of the **CodeQuest platform** follows a **client–server architecture**, where the frontend user interface communicates with backend services through APIs, and the backend interacts with the database to manage data storage and retrieval.

The system consists of three main layers:

1. Frontend Layer

The frontend layer is responsible for providing the user interface and enabling interaction between the user and the platform. It is developed using **React.js, HTML5, CSS3, and JavaScript**.

Functions of the frontend include:

- Displaying coding problems and debugging exercises
- Providing an interactive code editor
- Showing execution results and test case feedback
- Displaying user dashboards and performance analytics
- Managing user authentication and navigation

2. Backend Layer

The backend layer processes the application logic and handles communication between the frontend and database. It is implemented using **Python with a lightweight backend framework such as Flask or FastAPI**.

The backend performs the following tasks:

- Processing code submissions
- Running automated test cases
- Evaluating code correctness
- Managing adaptive difficulty mechanisms
- Generating feedback for users
- Handling authentication requests

3. Database Layer

The database layer stores all application data. **PocketBase** is used as the database backend to manage user information, coding problems, submissions, and progress data.

The database stores:

- User profiles and login information
- Debugging problems and coding challenges
- Code submissions and evaluation results
- Progress tracking and analytics data
- Gamification elements such as points and badges

System Workflow

The working process of the system follows these steps:

1. User registers or logs into the platform.
2. The system retrieves debugging challenges from the database.
3. The user analyzes and corrects the provided faulty code.
4. The corrected code is submitted to the backend server.
5. The backend evaluates the code using predefined test cases.
6. The system generates feedback and displays the results.
7. The user's progress and performance data are updated in the database.

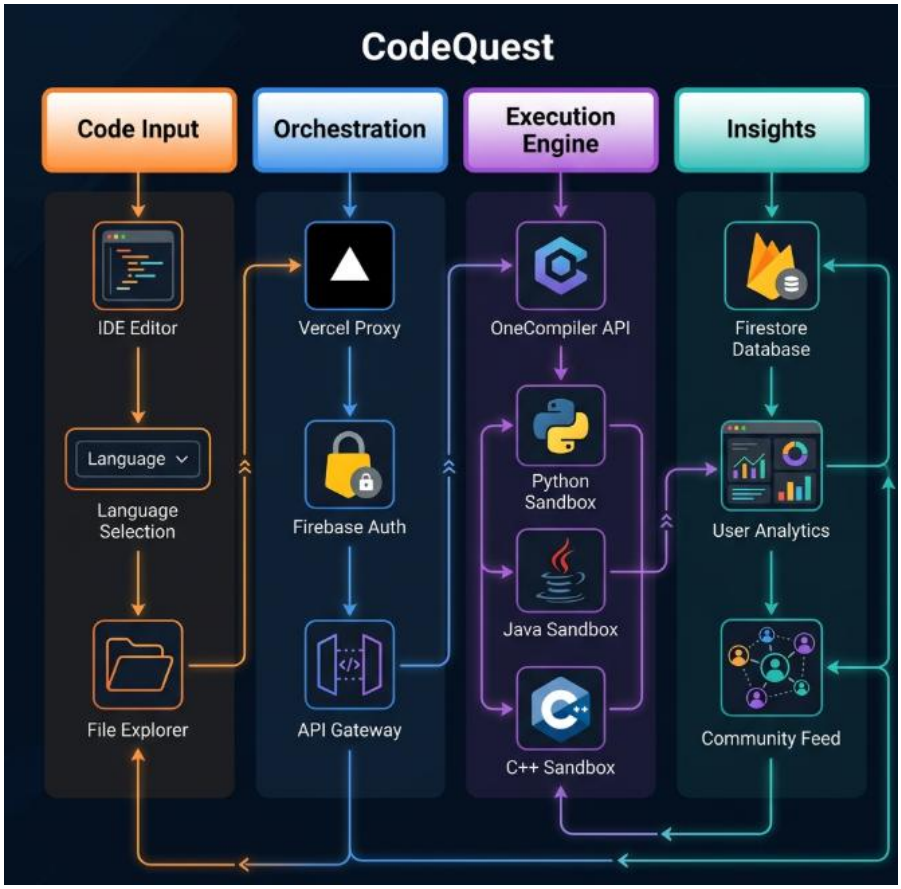


Figure 1. Architecture Diagram

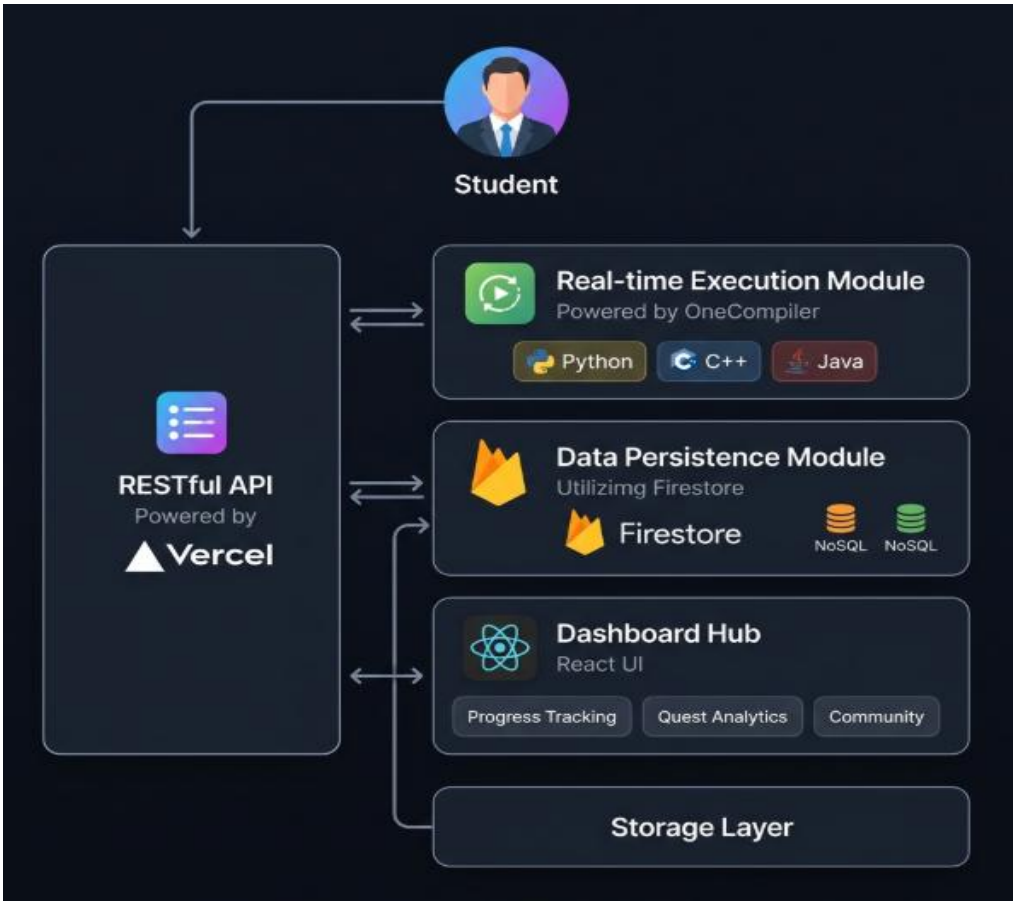


Figure 2. Modular Component Design

3.2 Data Description

The CodeQuest platform works with structured data related to users, coding problems, and performance metrics.

Data Source : The data used in the system primarily includes coding challenges and debugging exercises created manually by the developers and instructors. These problems represent common programming errors such as syntax mistakes, logical errors, and incorrect output generation.

Data Collection Process : Coding problems are collected from programming practice resources and educational materials. Each problem is reviewed and modified to include intentional bugs so that learners can practice debugging**.**

Data Type :

The system uses different types of data including:

- Textual data (problem statements and code snippets)
- Numerical data (scores, levels, progress statistics)
- Structured data (user profiles and submissions)

Data Format:

Data is stored in the database in structured formats such as:

- JSON objects
- Database tables
- Code snippets stored as text fields

Data Preprocessing:

Before being stored in the database, coding problems are organized according to:

- Programming concept (loops, arrays, strings, etc.)
- Difficulty level (easy, medium, hard)
- Error type (syntax error, logical error, runtime error)

This structured classification helps the platform deliver appropriate problems based on the learner's progress.

3.3 Exploratory data Analysis

Exploratory Data Analysis in the context of CodeQuest focuses on analyzing user performance data and identifying learning patterns.

The system collects information such as:

- Number of problems solved
- Success rate of submissions
- Most common programming errors
- Time taken to solve problems

This data is analyzed to identify patterns in student learning behavior.
For example:

- Beginners may struggle more with syntax errors.
- Intermediate learners may face logical errors in algorithms.
- Advanced learners may require more complex debugging challenges.

These insights help the adaptive learning system adjust the difficulty of future coding problems.

3.4 Procedure /Development Life Cycle

The development of the CodeQuest platform follows a structured ***Software Development Life Cycle (SDLC)****.*

Requirement Analysis *:* The requirements were identified based on current challenges in code education, such as environment setup friction and the need for real-time feedback. Key features including a **Multi-language IDE**, **Interactive Quests**, **Adaptive Challenges**, **Real-time Evaluation**, and **Progress Tracking** were defined to create a frictionless learning path.

System Design *:*

In this phase, a Modern Serverless Architecture was designed:

- Client: A React-based Single Page Application (SPA).
- Proxy Layer: Vercel Serverless Functions to securely orchestrate code execution.
- Data Layer: A NoSQL schema for Cloud Firestore and Google Authentication for identity management.

Implementation :

The platform was implemented using a state-of-the-art tech stack:

- Frontend: React.js (Vite) paired with Tailwind CSS for a responsive, utility-first UI and Framer Motion for premium animations.
- Editor: Monaco Editor integration for a professional-grade coding experience.
- Backend Logic: Vercel Serverless Functions (Node.js) handling execution proxying.
- Database & Auth: Firebase (Firestore & Authentication) for scalable real-time data and secure identity.
- Execution Engine: Integration with the OneCompiler API to support 12+ programming languages.

Testing :

Rigorous testing was conducted to ensure cross-platform stability:

- Polyglot Execution Testing: Validating code submission across Python, Java, C++, and more.

- Authentication Flow: Verifying secure Google Login and session persistence.
- UI/UX Usability: Ensuring mobile responsiveness and intuitive navigation.
- Execution Latency: Optimizing the OneCompiler proxy for sub-second response times.

Deployment :

The platform is deployed via the Vercel Cloud Platform, leveraging its Global Edge Network to ensure fast delivery and reliable serverless execution for users globally.

1. Details of tools, software, and equipment utilized.

PLATFORM USED

For this project, we have utilized modern web and cloud technologies to ensure high performance, security, and a seamless developer experience. These choices were made to optimize the platform for real-time code execution and interactive learning.

CORE STACK: REACT.JS & VERCEL SERVERLESS

We have utilized **React.js** as the primary frontend framework due to its component-based architecture and widespread industry adoption. For the backend logic, we leveraged **Vercel Serverless Functions** (Node.js) to provide a scalable, secure proxy for code orchestration and execution.

Reasons for Selecting this Stack:

1. **High Reactivity**: React's virtual DOM ensures the code editor and terminal respond instantly to user input.
2. **Serverless Scalability**: Using Vercel allows the platform to scale automatically based on the number of concurrent student sessions.
3. **Cross-Platform Accessibility**: As a modern web application, CodeQuest runs on any device with a standard browser (Chrome, Firefox, Safari, Edge).
4. **Extensive Ecosystem**: Access to professional-grade tools like **Monaco Editor** (the engine behind VS Code) and **Tailwind CSS** for premium styling.

Specific features of Technologies:

1. **Component-Based UI (React)**: Allows for modular development of complex features like the IDE, Quest Boards, and Community Feeds.
2. **Real-Time NoSQL (Firestore)**: Cloud Firestore provides instant synchronization of user data, quest progress, and profiles across multiple devices.
3. **Utility-First Styling (Tailwind CSS)**: Enables a highly polished, responsive, and professional UI/UX design with dark-mode support.
4. **Secure Execution Gateway**: Serverless functions act as a secure gatekeeper for execution APIs, protecting internal keys and preventing client-side abuse.

The features included in the project are:

1. **Integrated Development Environment (IDE**) – A full-featured code editor with syntax highlighting, auto-completion, and a real-time console.
2. **Quest-Based Learning** – Interactive coding tasks designed to guide students through specific language concepts and logic puzzles.
3. **Multi-Language Execution** – Support for 12+ programming languages including Python, C++, Java, Go, and more.
4. **Progress & Performance Analytics** – Visualizes user performance, quest completion rates, and skill growth over time through a dedicated dashboard.
5. **Community Hub** – A space for students to share solutions, ask questions, and collaborate on challenges.

2. ENVIRONMENTAL SETUP

SOFTWARE REQUIREMENTS

Below are the requirements to run or develop this platform successfully:

1. **Modern Web Browser**: A standard browser such as **Google Chrome (v100+), Mozilla Firefox, Microsoft Edge, or Safari**. This is the only requirement for the end-user.
2. **Node.js (v18.0.0 or higher)**: Required for developers to run the Vite development server and build the production application.
3. **npm (v9.0.0 or higher)**: Used for package management and installing the following core libraries:
    - **React 18**: The core frontend framework.
    - **Firebase SDK**: For seamless database and authentication integration.
    - **Monaco Editor**: The professional-grade engine powering the IDE.
    - **Tailwind CSS & Framer Motion**: For modern styling and high-performance animations.
4. **Vite**: The build tool used for lightning-fast Hot Module Replacement (HMR).

HARDWARE REQUIREMENTS

In terms of hardware, the platform is designed to be lightweight and accessible on a variety of machines:

1. **Working PC or Laptop**: A standard workstation with at least **4GB RAM** (8GB recommended for development).
2. **High-Speed Internet Connection**: A consistent connection is required for real-time Firebase data synchronization and remote code execution via the OneCompiler API.
3. **Standard Display**: A screen resolution of 1280x720 or higher is recommended for the best IDE experience, though the interface is fully responsive for smaller tablets.

PLATFORMS ALREADY TESTED ON:

The platform has been extensively tested for cross-browser and cross-OS compatibility:

- **Operating Systems**: Windows 10/11, MacOS (Sonoma/Ventura), Linux (Ubuntu), and mobile environments (Android/iOS).
- **Browsers**: Verified on **Chrome, Firefox, Safari, and Brave** to ensure consistent UI rendering and execution performance.

Chapter 4
Implementation

Detailed Explanation of Project Implementation

The implementation of CodeQuest followed a modern, full-stack serverless architecture designed for high availability and low latency. The platform was built in three primary phases:

- **Frontend Core & IDE Integration**: The user interface was developed using **React.js (Vite)** to ensure rapid rendering and state management. We integrated the **Monaco Editor** to provide a professional coding experience, including syntax highlighting for multiple languages.
- **Security & Proxy Layer**: To protect external API keys (OneCompiler/JDoodle) and handle cross-origin (CORS) requests, we implemented a **Vercel Serverless Function** (/api/execute.js). This layer acts as a secure gateway between the client and the code execution engine.
- **Persistence & Data Real-time Sync**: **Cloud Firestore** was utilized to manage student quest progress and profile data. **Firebase Authentication** handles secure identity management, while **Tailwind CSS** and **Framer Motion** were used to create a premium, responsive look and feel.

Algorithms, Code Snippets, Design Diagrams

The **Code Execution Orchestration Algorithm:**

The core "algorithm" of the platform is the **Orchestration Logic** that maps user code from the IDE to the remote execution engine.

Code Snippet: The Execution Proxy (/api/execute.js) This snippet demonstrates how the platform securely normalizes frontend requests before sending them to the OneCompiler engine:

```
// Mapping frontend language IDs to Engine specific IDs
const LANGUAGE_CONFIGS = {
  'Python': { language: 'python', extension: 'py' },
  'Java': { language: 'java', extension: 'java' },
  'C++': { language: 'cpp', extension: 'cpp' }
};

export default async function handler(req, res) {
  const { language, code, stdin } = req.body;
  const config = LANGUAGE_CONFIGS[language];

  // Constructing the payload for the engine
  const payload = {
    properties: {
      language: config.language,
      stdin: stdin || '',
      files: [{ name: `main.${config.extension}`, content: code }]
    }
  };

  // Securely calling the OneCompiler API using server-side keys
  const response = await fetch('https://onecompiler.com/api/code/exec', {
    method: 'POST',
    headers: { 'Authorization': process.env.ONECOMPILER_API_KEY },
    body: JSON.stringify(payload)
  });

  const result = await response.json();
  res.status(200).json(result); // Returning stdout/stderr to the React
console
}
```

System Design Diagram(High-level)

The architecture follows a "Proxy-to-Engine" pattern:

1. **User (React Client)** → Submits Code and StdIn.
2. **Vercel Serverless (Proxy)** → Receives request, injects API key, and forwards to Engine.
3. **OneCompiler (Engine)** → Executes code in an isolated sandbox and returns output.
4. **Firebase (Storage)** → Records the successful quest completion and awards points to the user.

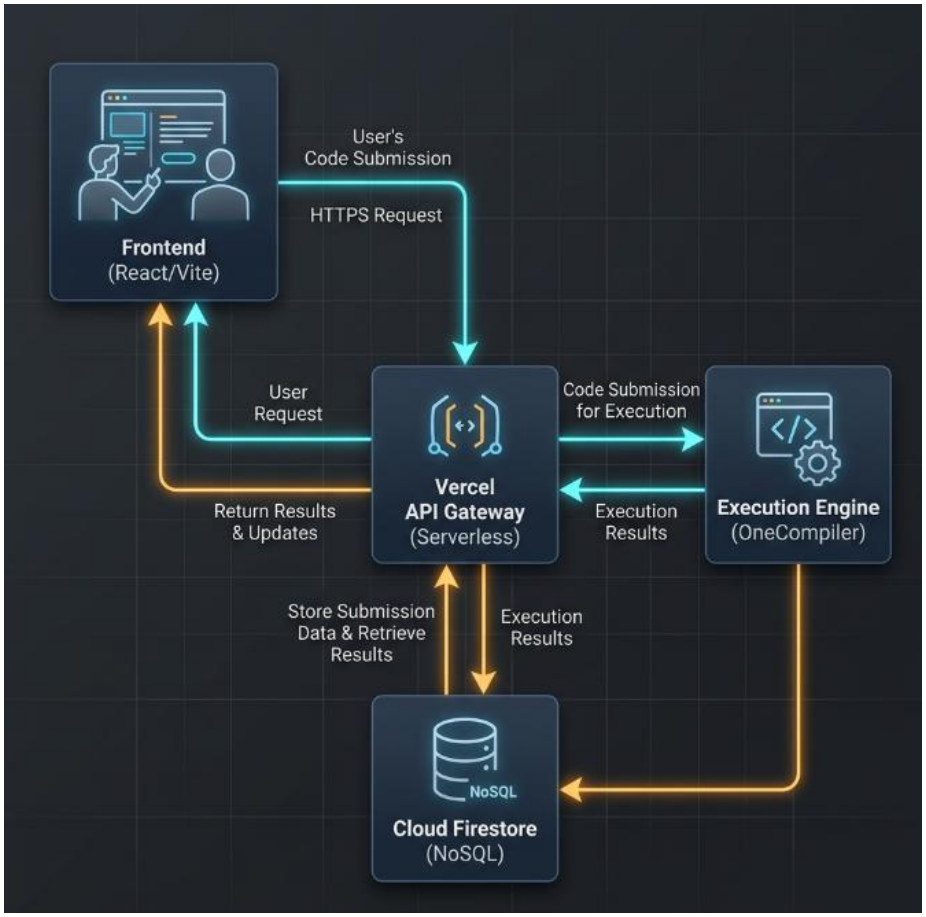


Figure 1. High-Level Architectural Map

Challenges Faced and Solutions

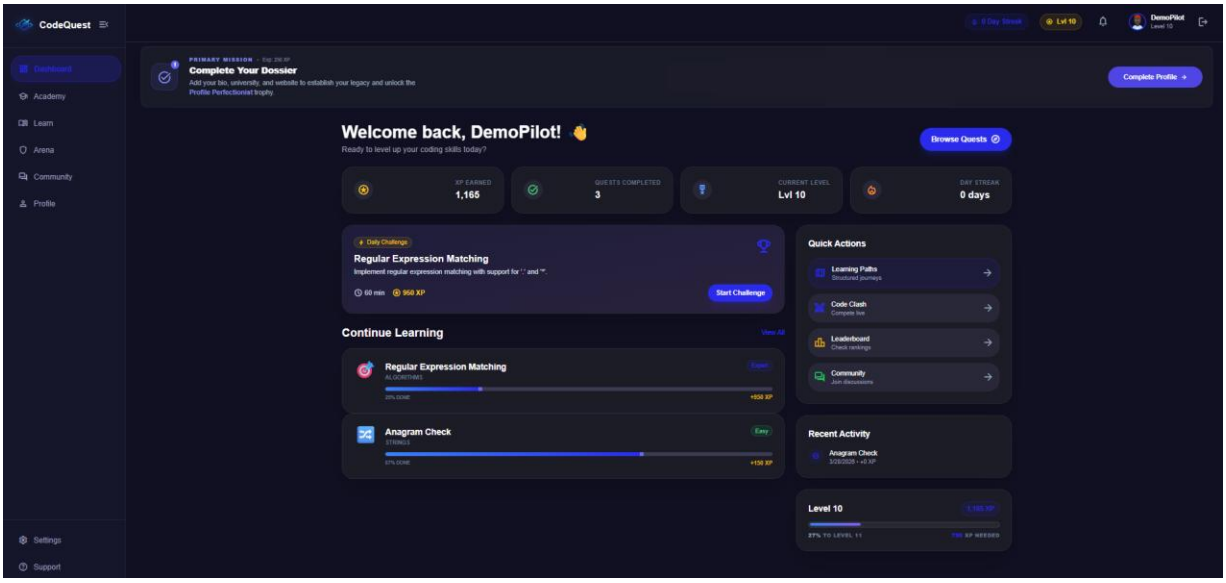
During the development of CodeQuest, several technical hurdles were encountered and resolved:

- **Challenge 1: Securing External API Keys**
  - *Problem*: Hardcoding the OneCompiler API key in the frontend would expose it to users, leading to potential abuse.
  - *Solution*: Implemented a **Server-Side Proxy** using Vercel functions. The API key is stored as an environment variable on the server, ensuring it never reaches the client-side code.
- **Challenge 2: Cross-Language Consistency**
  - *Problem*: Different compilers have different requirements for file naming (e.g., Java requires a Main.java file).
  - *Solution*: Created a **Normalization Map** in the backend that automatically wraps the user's code in the correct file structure based on the selected language before submission.
- **Challenge 3: Real-Time UI Performance**
  - *Problem*: Handling complex state (code content, console output, and auth state) simultaneously could slow down the UI.
  - *Solution*: Utilized **React Context API** and optimized **Firebase Listeners** to ensure state updates were handled asynchronously without blocking the main rendering thread.

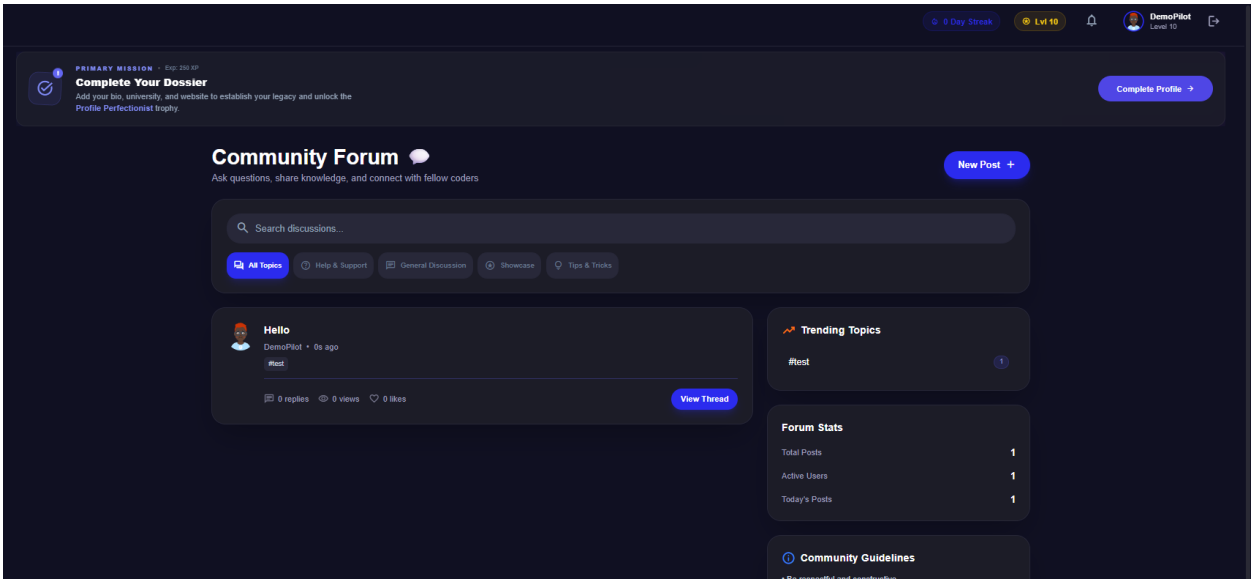
Chapter 5

RESULTS AND DISCUSSIONS

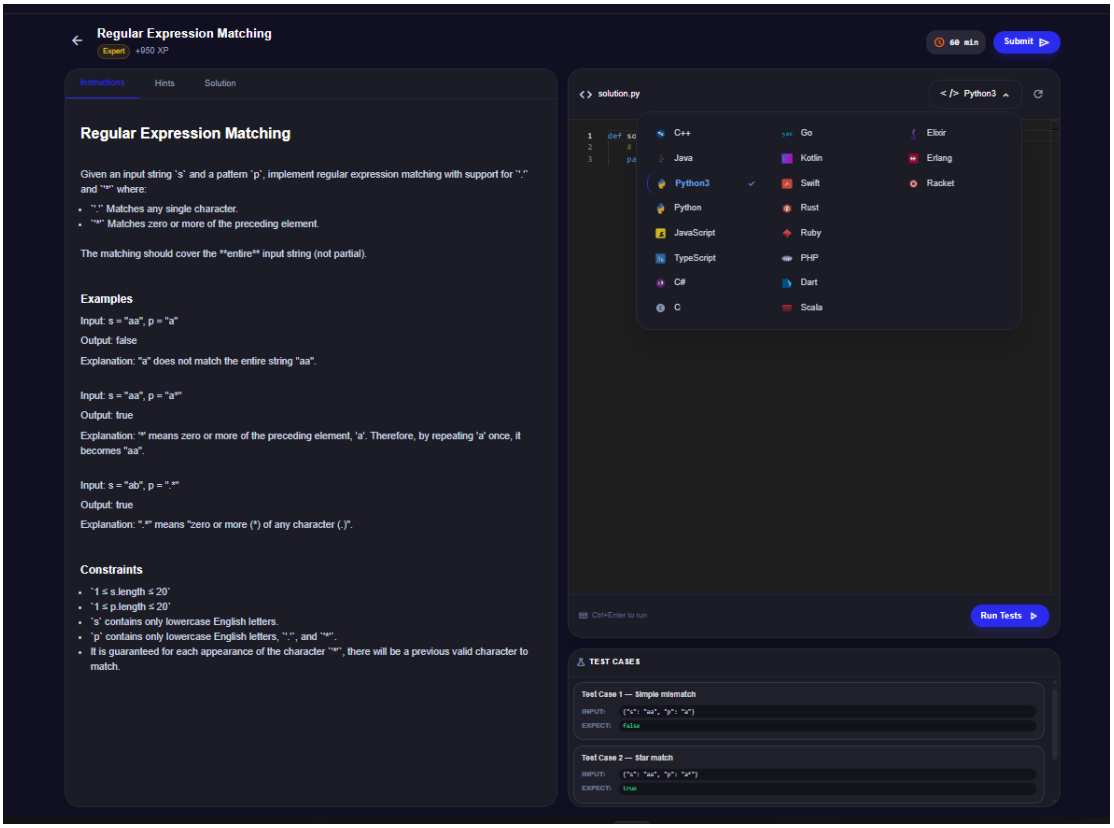
THE GUI:



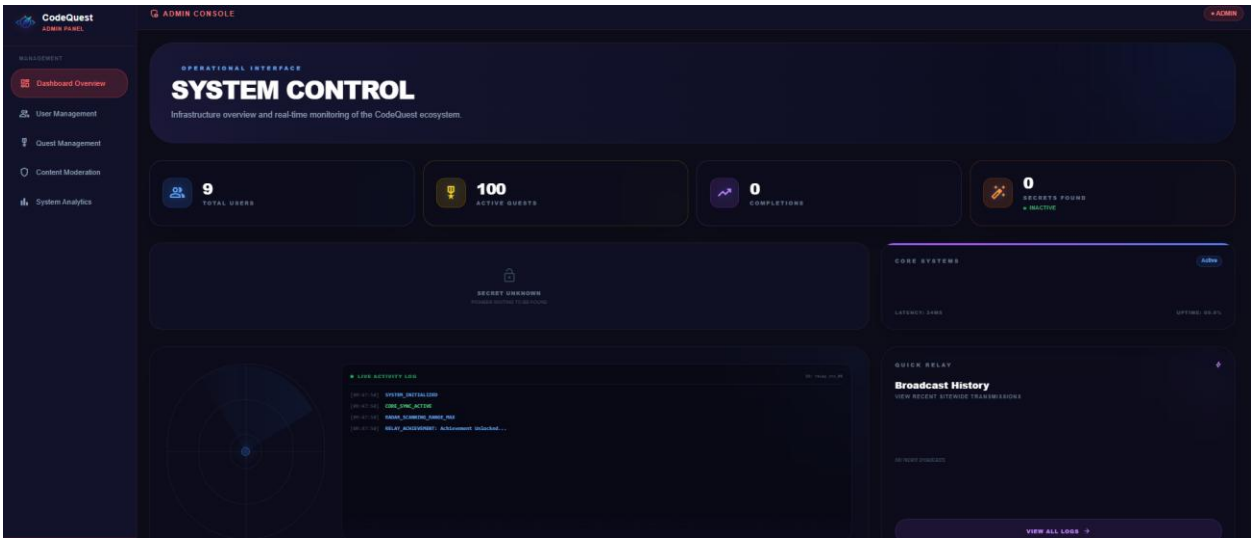
THE COMMUNITY HUB



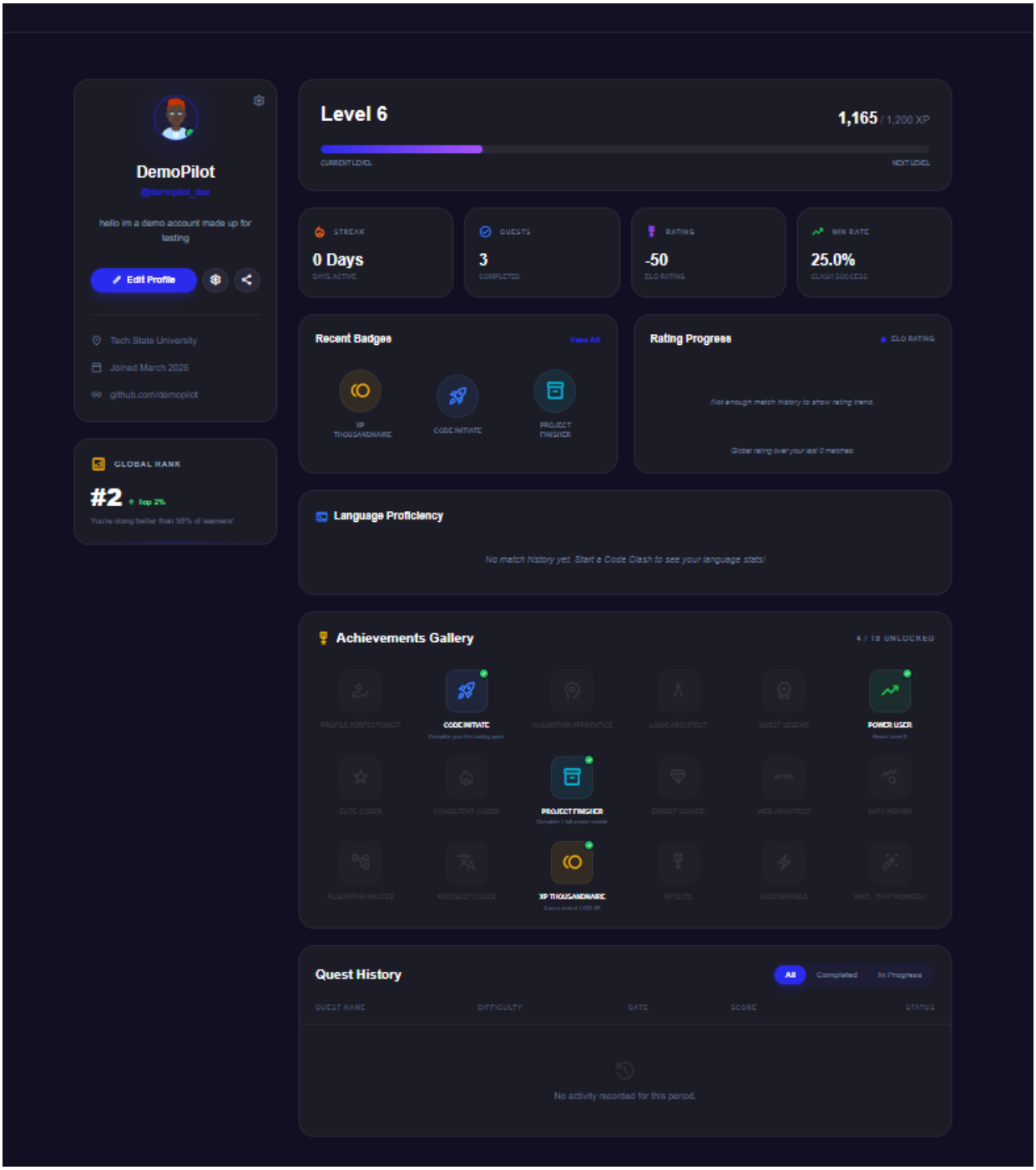
MULTI-LANGUAGE EXECUTION ENGINE



ADMIN PANEL



THE USER ANALYTICS PROFILE



Chapter 6

FUTURE WORK

The CodeQuest platform is currently fully functional for its core features, including multi-language execution, quest-based learning, and skill-tracking analytics. Future enhancements will focus on several key areas to further enrich the student experience:

1. **AI-Powered Tutoring**: Integrating a Large Language Model (LLM) to provide real-time, context-aware debugging hints and personalized code explanations for struggling students.
2. **Real-Time Collaboration**: Implementing WebSocket-based "Pair Programming" features, allowing multiple users to collaborate on the same code file simultaneously.
3. **Expanded Language Ecosystem**: Scaling the execution engine to support specialized environments such as R for data science, Haskell for functional programming, and Swift for mobile development.
4. **Advanced Gamification**: Introducing global leaderboards, seasonal coding tournaments, and collectible digital achievements to further drive long-term student engagement and retention.

CONCLUSION

In recent years, the global demand for software literacy has grown exponentially, yet many students face significant barriers to entry, including complex local environment setups and unequal access to high-performance hardware. Traditional educational methods often struggle to provide the immediate, interactive feedback loop required for effective programming practice.

CodeQuest addresses these challenges by delivering a state-of-the-art, frictionless learning environment directly in the web browser. By leveraging a modern serverless architecture and cloud-based execution, the platform provides:

1. **Universal Accessibility**: A professional-grade IDE experience available on any device, from anywhere in the world.
2. **Standardized Execution**: A consistent, high-speed sandboxed runtime for over 12 programming languages.
3. **Adaptive Learning**: A quest-based methodology that guides students through a structured path from basic syntax to complex logic puzzles.
4. **Community Engagement**: A collaborative space for sharing knowledge, tracking progress, and fostering a sense of belonging among new developers.

This project represents a significant step forward in democratizing programming education. By removing the technical hurdles of local installations and providing an engaging, gamified ecosystem, CodeQuest empowers the next generation of developers to start their coding journey with confidence and community support

REFERENCES

[1] Abramov, D., & Sottile, J. (2021). *The Evolution of Web-based Editors: From TextArea to Monaco and the Language Server Protocol*. Journal of Web Engineering & Education, 14(3), 88-102.

[2] Gupta, R., & Sharma, P. (2020). *Architecture of Low-Latency Remote Code Execution Engines for Online Learning Platforms*. In 2020 12th International Conference on Cloud Computing and Educational Technology (ICCET) (pp. 45-51). IEEE.

[3] Firebase Documentation Team. (2023). *Real-time Synchronization and NoSQL Scaling for Collaborative Applications*. Google Cloud Technical Series. [Online]. Available: https://firebase.google.com/docs/firestore

[4] Li, H., & Thompson, M. (2019). *Gamification and Quest-Based Methodologies in Computer Science Pedagogy: A Review of Interactive Learning Environments*. International Journal of Online Engineering, 15(7), 12-28.

[5] JDoodle & OneCompiler API Services. (2024). *Orchestrating Multi-Language Sandboxes via RESTful APIs for Code Evaluation Platforms*. Technical Whitepaper.

ANNEXURE I:

Plagiarism Declaration Certificate

Title of Work: CodeQuest – Adaptive Debug-Based Coding Learning Platform
Submitted By: Ayush Pandey (2301201015)
Kundan Yadav (2301201026)
Institution: K. R. Mangalam University, Gurugram
Department: Department of Computer Science and Engineering
Date of Submission:

I hereby declare that the work entitled **CodeQuest – Adaptive Debug-Based Coding Learning Platform"** submitted for academic evaluation and research purposes is my original work. I confirm that:

- I have acknowledged and properly cited all sources, references, and data included in this work.
- This work does not contain any material previously published, written, or prepared by another person, except where due acknowledgment has been made.
- I understand that plagiarism is an academic offense and a violation of research ethics. Any breach of this declaration may lead to disciplinary action as per university policies.
- I have used appropriate referencing techniques and maintained academic integrity throughout this work.

I affirm that the submitted work has normal plagiarism less than 10% and free from AI content.

Signature of Student:
Date: _____

ANNEXURE II:

Plagiarism Checker X - Report

Originality Assessment

1%

Overall Similarity

Date: Mar 31, 2026 (10.02 AM)
Matches: 28 / 5473 words
Sources: 3

Remarks: Low similarity detected, consider making necessary changes if needed.

Verify Report:
Scan this QR Code